Object pooling: pool resources, not small objects

A pool pays off for costly objects; for small ones, allocate fewer instead.

- Pool threads, connections and direct buffers.
- The JDK already pools `Integer` -128 to 127 through `Integer.valueOf`.
- For small objects: presize collections, use primitives, avoid boxing.
- A pooled object lives long, gets promoted, and needs locking.

Enabling compact object headers


```
java -XX:+UseCompactObjectHeaders -jar app.jar
jcmd <pid> VM.flags -all | grep UseCompactObjectHeaders     # check that it is on
```

Retained size	Off	On	Saved
`HashMap<Integer, Integer>`, 1 M entries	56.4 MB	48.4 MB	14%
`ArrayList<Integer>`, 1 M, presized	20.0 MB	20.0 MB	0%
`Stream.filter().map().mapToLong()`	304 B	272 B	11%

- One flag, no code change: it shrinks nodes, not boxes.

Non-capturing lambdas


```java
// capturing: a new lambda on every call, holding the value
Iterate.anySatisfy(list, Integer.valueOf(test)::equals);

// fat-free: one constant lambda, the value is a parameter
Iterate.anySatisfyWith(list, Integer::equals, test);
```

- Escape analysis off: 32 B capturing, 16 B fat-free (only the `Integer` box).
- Move captured values into parameters: the `*With` methods.

Eclipse Collections: primitives and no pipeline

	JDK	Eclipse Collections
1 M `int`s in a list	20.0 MB `ArrayList<Integer>`	4.0 MB `IntArrayList`
1 M `int`s in a set	48.4 MB `HashSet<Integer>`	8.4 MB `IntHashSet`
`anyMatch`, match at 500	176 B per call	0 B `anySatisfy`
`filter().count()`	224 B per call	0 B `count`

- Primitive collections drop the boxes; eager methods loop without a pipeline.
- `*With` methods keep the lambda fat-free.